Formalización Matemática, Demostraciones de NP-Dureza y Análisis de Complejidad Asintótica

Proyecto Semestral ADAG — Grupo 3 (Ruta)
Primera Entrega: Aceleración Óptima bajo Presupuesto y Cobertura de Auditoría

Grupo 3 (Ruta)
Juan José Hernández Velásquez
Victor Manuel Valencia Alvarez

Análisis y Diseño de Algoritmos (ADAG)
Facultad de Ingeniería — Periodo 2026-2

5 de noviembre de 2026

Resumen

Este informe técnico presenta la fundamentación matemática y teórica rigurosa para la Primera Entrega del Proyecto Semestral de Análisis y Diseño de Algoritmos (ADAG). Se formalizan íntegramente las dos problemáticas asignadas al Grupo 3 (Ruta): (1) la *Aceleración Óptima de una Cadena de Flujo en Serie bajo Presupuesto*, demostrando su equivalencia estricta con el problema de la Mochila 0/1, su NP-dureza por reducción polinomial y su resolución exacta mediante Programación Dinámica en tiempo pseudo-polinomial $\mathcal{O}(n \cdot B)$; y (2) la *Cobertura Mínima de Auditoría* (versión cardinal y variante ponderada por costo), demostrando su NP-dureza mediante una reducción polinomial formal desde *Vertex Cover*, deduciendo asintóticamente la complejidad $\mathcal{O}(n \cdot r^2)$ del algoritmo voraz y derivando formalmente la cota armónica de aproximación $H_r \leq \ln(r) + 1$. Adicionalmente, se describen la arquitectura de software reproducible implementada en el repositorio del equipo y los resultados verificados sobre el caso de prueba oficial de la profesora.

Índice

1. Introducción y Contextualización del Sistema **2**

2. Parte 1: Aceleración Óptima de la Cadena bajo Presupuesto **2**

2.1. Formalización Matemática del Problema 2
2.2. Demostración Formal de NP-Dureza por Reducción Polinomial 3
2.3. Algoritmo Exacto: Programación Dinámica 5
2.3.1. Subestructura Óptima y Ecuación de Recurrencia 5
2.3.2. Reconstrucción de la Solución y Backtracking 6
2.4. Análisis de Complejidad y Justificación de Pseudo-Polinomialidad 6
2.4.1. Justificación Formal del Carácter Pseudo-Polinomial 7

3. Parte 2: Cobertura Mínima de Auditoría **7**
3.1. Formalización Matemática del Problema 7
3.2. Demostración Formal de NP-Dureza por Reducción desde Vertex Cover 8
3.3. Diseño del Algoritmo Voraz y Variante Ponderada 10
3.3.1. Criterios de Selección Voraz 10
3.4. Deducción Paso a Paso de la Función de Eficiencia Asintótica 10
3.5. Demostración y Análisis de la Cota Armónica de Aproximación 11

4. Resultados sobre el Caso de Prueba Oficial **13**
4.1. Descripción del Caso de Prueba 13
4.2. Resultados: Aceleración Óptima (DP Mochila 0/1) 13
4.3. Resultados: Cobertura Mínima de Auditoría 14
4.3.1. Voraz Estándar (Cardinalidad Mínima) 14
4.3.2. Voraz Ponderado por Costo (Variante 1 – Chvátal 1979) 14
4.4. Verificación del Presupuesto Conjunto 14
4.5. Resumen de Tiempos de Ejecución 14

5. Arquitectura del Repositorio y Trazabilidad de Resultados **15**
5.1. Separación Estricta de Responsabilidades (SoC) 15
5.2. Estructura de Trazas Generadas 15

6. Conclusiones **16**

1. Introducción y Contextualización del Sistema

En la arquitectura de sistemas empresariales y plataformas transaccionales modernas, el procesamiento de solicitudes críticas requiere armonizar dos vectores contrapuestos: la **eficiencia temporal** del flujo de trabajo (minimizar la latencia acumulada de procesamiento) y la **mitigación de riesgos normativos** (auditar interacciones para garantizar el cumplimiento de directrices legales, éticas y de ciberseguridad).

El presente informe aborda de manera formal el desacoplamiento y análisis algorítmico de estos dos pilares:

1. **Aceleración Óptima de la Cadena bajo Presupuesto:** Un proceso secuencial compuesto por n etapas en serie, donde cada etapa puede ser acelerada mediante una inversión de capital fija sujeta a un presupuesto global B.
2. **Cobertura Mínima de Auditoría:** La selección estratégica de un subconjunto de interacciones candidatas que cubra todas las clases de riesgo identificadas en el sistema, minimizando la cardinalidad o el costo total incurrido.

2. Parte 1: Aceleración Óptima de la Cadena bajo Presupuesto

2.1. Formalización Matemática del Problema

Definición 1 (Tamaño de la Entrada) *La instancia del problema viene parametrizada por la tupla (n, B), donde:*

- $n \in \mathbb{Z}^+$ *representa el número total de etapas secuenciales del pipeline.*
- $B \in \mathbb{Z}^+ \cup \{0\}$ *representa el presupuesto financiero disponible expresado en unidades enteras discretas.*

Definición 2 (Conjunto de Entradas) *El problema recibe formalmente:*

- *Un conjunto ordenado de etapas $E = \{1, 2, \ldots, n\}$.*
- *Para cada etapa $i \in E$, una terna de parámetros enteros estrictamente positivos:*

$$(t_i, c_i, d_i) \in \mathbb{Z}^+ \times \mathbb{Z}^+ \times \mathbb{Z}^+$$

  donde:

  - *t_i: Duración nominal de la etapa sin aceleración.*
  - *c_i: Costo financiero de aplicar la aceleración a la etapa i.*
  - *d_i: Reducción de tiempo alcanzada al acelerar la etapa i.*

  Se satisface la invariante de dominio estricta $t_i > d_i > 0$ y $c_i > 0$.

- *Un límite de presupuesto $B \geq 0$.*

Definición 3 (Espacio de Salidas) *Un vector de decisiones binarias* $X = (x_1, x_2, \ldots, x_n) \in \{0,1\}^n$*, donde:*

$$x_i = \begin{cases} 1 & \textit{si la etapa } i \textit{ es acelerada,} \\ 0 & \textit{en caso contrario.} \end{cases}$$

A partir de X*, se definen los indicadores agregados:*

- *Costo total invertido:* $C_{total}(X) = \sum_{i=1}^{n} c_i x_i$.
- *Duración final de la cadena:* $T_{final}(X) = \sum_{i=1}^{n} (t_i - d_i x_i)$.
- *Ahorro total de tiempo:* $\Delta T(X) = \sum_{i=1}^{n} d_i x_i$.

Definición 4 (Función Objetivo y Demostración de Equivalencia) *Dado que las* n *etapas están dispuestas estrictamente en serie, el camino crítico está compuesto por la suma total de las duraciones de todas las etapas. Por ende, el objetivo primordial es:*

$$\min_{X \in \{0,1\}^n} T_{final}(X) = \min_{X \in \{0,1\}^n} \sum_{i=1}^{n} (t_i - d_i x_i)$$

Dado que $\sum_{i=1}^{n} t_i$ *es una constante invariante para una instancia fija del pipeline:*

$$\min_{X \in \{0,1\}^n} \left[\sum_{i=1}^{n} t_i - \sum_{i=1}^{n} d_i x_i \right] \equiv \max_{X \in \{0,1\}^n} \sum_{i=1}^{n} d_i x_i.$$

Esto demuestra que minimizar la duración total de la cadena en serie es matemáticamente idéntico a maximizar la suma de reducciones de tiempo logradas.

Definición 5 (Restricciones del Modelo) *El vector de decisión debe satisfacer:*

$$\sum_{i=1}^{n} c_i x_i \leq B \tag{1}$$

$$x_i \in \{0,1\}, \quad \forall i \in \{1, \ldots, n\}. \tag{2}$$

2.2. Demostración Formal de NP-Dureza por Reducción Polinomial

Para probar formalmente que el problema de **Aceleración de Cadena bajo Presupuesto** (ACP) es NP-difícil, consideramos su versión de decisión asociada y construimos una reducción desde el problema clásico de la **Mochila 0/1** (*0/1 Knapsack Problem*), reconocido como NP-completo clásico (Karp, 1972).

Definición 6 (Problema de Decisión de la Mochila 0/1) *Dada una colección de* m *objetos con pesos* $w_j \in \mathbb{Z}^+$*, valores* $v_j \in \mathbb{Z}^+$*, una capacidad máxima* $W \in \mathbb{Z}^+$ *y un umbral de valor* $V^* \in \mathbb{Z}^+$*, ¿existe un subconjunto* $S \subseteq \{1, \ldots, m\}$ *tal que:*

$$\sum_{j \in S} w_j \leq W \quad y \quad \sum_{j \in S} v_j \geq V^* \ ?$$

Definición 7 (Problema de Decisión de Aceleración bajo Presupuesto (ACP-D)) *Dada una cadena de n etapas con ternas (t_i, c_i, d_i), un presupuesto $B \in \mathbb{Z}^+$ y una cota superior de duración total permitida $K \in \mathbb{Z}^+$, ¿existe un vector $X \in \{0,1\}^n$ tal que:*

$$\sum_{i=1}^{n} c_i x_i \leq B \quad y \quad \sum_{i=1}^{n} (t_i - d_i x_i) \leq K \ ?$$

Teorema 1 *El problema de Aceleración de Cadena bajo Presupuesto (ACP-D) es NP-completo, y su versión de optimización asociada es NP-difícil.*

Demostración:

La demostración procede en dos partes: pertenencia a la clase NP y NP-dureza por reducción.

1. Pertenencia a NP: Dado un certificado $X = (x_1, \ldots, x_n) \in \{0,1\}^n$:

- Evaluar $\sum_{i=1}^{n} c_i x_i$ requiere n multiplicaciones y sumas, verificando si es $\leq B$ en tiempo $\mathcal{O}(n)$.
- Evaluar $\sum_{i=1}^{n} (t_i - d_i x_i)$ toma tiempo $\mathcal{O}(n)$, verificando si es $\leq K$.

Dado que el proceso de verificación toma tiempo $\mathcal{O}(n)$, polinomial en el tamaño de la entrada, ACP-D $\in$ NP.

2. Reducción Polinomial desde Mochila 0/1 (Mochila 0/1 $\leq_P$ ACP-D):
Sea $I_{KP} = (\{w_j\}_{j=1}^{m}, \{v_j\}_{j=1}^{m}, W, V^*)$ una instancia arbitraria de la Mochila 0/1. Construimos una instancia equivalente $I_{ACP} = (\{(t_i, c_i, d_i)\}_{i=1}^{n}, B, K)$ mediante el siguiente mapeo determinista:

1. Fijamos el número de etapas $n = m$.
2. Para cada objeto $j \in \{1, \ldots, m\}$, definimos la etapa $i = j$ con:

$$\begin{aligned} c_i &= w_j \quad \text{(el costo de aceleración equivale al peso del objeto)}, \\ d_i &= v_j \quad \text{(la reducción de tiempo equivale al valor del objeto)}, \\ t_i &= v_j + 1 \quad \text{(duración nominal } t_i > d_i > 0 \text{ estrictamente válida)}. \end{aligned}$$

3. Fijamos el presupuesto disponible $B = W$.
4. Fijamos la cota de duración máxima:

$$K = \sum_{i=1}^{n} t_i - V^* = \sum_{j=1}^{m} (v_j + 1) - V^*.$$

Esta transformación construye n ternas y 2 constantes escalares. Cada asignación toma tiempo $\mathcal{O}(1)$, por lo que la función de reducción se computa en tiempo y espacio estrictamente $\mathcal{O}(n)$, lo cual es lineal y por tanto polinomial en la longitud de I_{KP}.

3. Demostración de Equivalencia (Doble Implicación):
Demostraremos que I_{KP} es una instancia positiva si y solo si I_{ACP} es una instancia positiva.

($\Rightarrow$) Supongamos que existe una solución factible $S \subseteq \{1, \ldots, m\}$ para I_{KP} tal que $\sum_{j \in S} w_j \leq W$ y $\sum_{j \in S} v_j \geq V^*$.
Definimos el vector $X \in \{0,1\}^n$ asignando $x_i = 1$ si $i \in S$ y $x_i = 0$ si $i \notin S$.
Verificamos las restricciones de I_{ACP}:

- Presupuesto:
$$\sum_{i=1}^{n} c_i x_i = \sum_{j \in S} w_j \leq W = B.$$
- Duración máxima de la cadena:
$$T_{final}(X) = \sum_{i=1}^{n} (t_i - d_i x_i) = \sum_{i=1}^{n} t_i - \sum_{i=1}^{n} d_i x_i = \sum_{i=1}^{n} t_i - \sum_{j \in S} v_j.$$

  Dado que $\sum_{j \in S} v_j \geq V^*$, restando a ambos lados se obtiene:
$$\sum_{i=1}^{n} t_i - \sum_{j \in S} v_j \leq \sum_{i=1}^{n} t_i - V^* = K.$$

Por lo tanto, X es un certificado válido que satisface I_{ACP}.

($\Leftarrow$) Supongamos que existe un vector $X \in \{0,1\}^n$ tal que $\sum_{i=1}^{n} c_i x_i \leq B$ y $T_{final}(X) \leq K$. Definimos el conjunto de objetos $S = \{j \in \{1, \ldots, m\} \mid x_j = 1\}$.

- Capacidad de la mochila:
$$\sum_{j \in S} w_j = \sum_{i=1}^{n} c_i x_i \leq B = W.$$
- Valor obtenido: Por hipótesis, $\sum_{i=1}^{n} t_i - \sum_{i=1}^{n} d_i x_i \leq K$. Sustituyendo la definición de K:
$$\sum_{i=1}^{n} t_i - \sum_{j \in S} v_j \leq \sum_{i=1}^{n} t_i - V^* \implies \sum_{j \in S} v_j \geq V^*.$$

Así, S es una solución factible para I_{KP}.

Habiendo demostrado validez, bicondicionalidad y costo temporal polinomial, se concluye que ACP-D es NP-completo y el problema de optimización asociado es NP-difícil. ■

Corolario 1 *Como una cadena de etapas en serie representa el caso particular más simple de un flujo general estructurado como un Grafo Acíclico Dirigido (DAG) con rutas alternativas y etapas concurrentes, el problema general de optimización temporal de flujos bajo presupuesto es al menos tan difícil como la cadena en serie, siendo por ende NP-difícil.*

2.3. Algoritmo Exacto: Programación Dinámica

Dada la correspondencia exacta con la Mochila 0/1, formulamos la solución óptima exacta mediante Programación Dinámica aplicando el Principio de Optimalidad de Bellman.

2.3.1. Subestructura Óptima y Ecuación de Recurrencia

Sea $DP[i][b]$ la máxima reducción de tiempo alcanzable considerando un subconjunto de las primeras i etapas ($1 \leq i \leq n$) bajo un presupuesto máximo disponible de b unidades ($0 \leq b \leq B$).

Casos Base:

$$DP[0][b] = 0, \quad \forall b \in \{0, 1, \ldots, B\}, \qquad DP[i][0] = 0, \quad \forall i \in \{0, 1, \ldots, n\}.$$

Relación de Recurrencia: Para $i \in \{1, \ldots, n\}$ y $b \in \{0, \ldots, B\}$:

$$DP[i][b] = \begin{cases} DP[i-1][b] & \text{si } c_i > b, \\ \max\Big(DP[i-1][b],\ DP[i-1][b-c_i] + d_i\Big) & \text{si } c_i \leq b. \end{cases}$$

2.3.2. Reconstrucción de la Solución y Backtracking

La reconstrucción óptima del vector $X = (x_1, \ldots, x_n)$ se realiza en tiempo $\mathcal{O}(n)$ descendiendo desde (n, B):

1. Para $i = n, n-1, \ldots, 1$:
   - Si $c_i \leq b$ y $DP[i][b] = DP[i-1][b-c_i] + d_i$ con $DP[i][b] > DP[i-1][b]$, entonces la etapa i fue acelerada en la solución óptima: $x_i = 1$, y el presupuesto remanente se actualiza $b \leftarrow b - c_i$.
   - En caso contrario: $x_i = 0$, conservando b.

2.4. Análisis de Complejidad y Justificación de Pseudo-Polinomialidad

Teorema 2 (Complejidad Temporal y Espacial) *El algoritmo de Programación Dinámica para la Aceleración de Cadena tiene una complejidad temporal de:*

$$T(n, B) = \Theta(n \cdot B)$$

y una complejidad espacial de $\Theta(n \cdot B)$ para almacenar la matriz de transiciones.

Demostración:

La tabla posee $(n+1)$ filas y $(B+1)$ columnas, totalizando $(n+1)(B+1)$ celdas. El cómputo de cada celda $DP[i][b]$ realiza una comparación condicional, una resta, una suma y una función máximo entre dos escalares, operaciones elementales que se ejecutan en tiempo $\mathcal{O}(1)$. Por ende, la fase de llenado de la matriz toma:

$$T_{llenado} = \sum_{i=1}^{n} \sum_{b=0}^{B} \Theta(1) = \Theta(n \cdot B).$$

La fase de reconstrucción ejecuta un bucle descendente desde n hasta 1. En cada paso se evalúa una celda en $\mathcal{O}(1)$, requiriendo un tiempo $\mathcal{O}(n)$. En consecuencia:

$$T(n, B) = \Theta(n \cdot B) + \mathcal{O}(n) = \Theta(n \cdot B).$$

■

2.4.1. Justificación Formal del Carácter Pseudo-Polinomial

Es indispensable clarificar por qué una función de eficiencia $\mathcal{O}(n \cdot B)$ no es polinomial en sentido estricto dentro de la teoría de la complejidad computacional:

- La longitud formal de una entrada en teoría de la computación se mide por la cantidad de bits necesarios para representarla en memoria.
- El número de etapas n utiliza $\mathcal{O}(n \log n)$ bits, pero el valor numérico escalar B se codifica en:

$$k = \lfloor \log_2 B \rfloor + 1 \text{ bits.}$$

- Si expresamos el costo computacional en términos de la longitud binaria k de la representación de B, tenemos que $B = 2^{\mathcal{O}(k)}$. Por consiguiente:

$$T(n, k) = \mathcal{O}(n \cdot 2^k).$$

Así, la función de eficiencia es **exponencial respecto a la cantidad de bits necesarios para codificar el presupuesto** B. Se denomina *pseudo-polinomial* porque se comporta polinomialmente solo si el valor escalar de B está acotado por un polinomio en n ($B \leq \text{poly}(n)$). Cuando B asume magnitudes numéricas arbitrariamente grandes, la matriz de programación dinámica satura exponencialmente tanto el tiempo de cómputo como la memoria primaria de la máquina.

3. Parte 2: Cobertura Mínima de Auditoría

3.1. Formalización Matemática del Problema

Definición 8 (Tamaño de la Entrada) *La instancia viene caracterizada por el par (n, r), donde:*

- *$r = |U|$ es el número total de categorías o clases de riesgo normativo identificadas en el sistema.*
- *$n = |\mathcal{S}|$ es el número de interacciones candidatas disponibles para auditoría.*

Definición 9 (Conjunto de Entradas)

- *Un universo finito de categorías de riesgo normativo:*

$$U = \{R_1, R_2, \ldots, R_r\}.$$

- *Una colección o familia de subconjuntos de riesgo $\mathcal{S} = \{S_1, S_2, \ldots, S_n\}$, donde cada $S_j \subseteq U$ representa el conjunto de riesgos mitigados al auditar la interacción j.*
- *Para la **Variante 1 (Ponderada)**, un vector de costos reales estrictamente positivos:*

$$W = (w_1, w_2, \ldots, w_n) \in \mathbb{R}^n_{>0},$$

donde w_j representa el costo operacional de ejecutar la auditoría de la interacción j. En el caso base no ponderado, $w_j = 1, \forall j$.

Definición 10 (Espacio de Salidas) *Un vector de selección binaria* $Y = (y_1, y_2, \ldots, y_n) \in \{0,1\}^n$, *donde:*

$$y_j = \begin{cases} 1 & \textit{si la interacción } S_j \textit{ es seleccionada para auditoría,} \\ 0 & \textit{en caso contrario.} \end{cases}$$

Definición 11 (Función Objetivo)

- ***Caso Base (Cobertura Cardinal Mínima):*** *Minimizar el número total de interacciones inspeccionadas:*

$$\min_{Y \in \{0,1\}^n} \sum_{j=1}^{n} y_j.$$

- ***Variante 1 (Cobertura Ponderada por Costo Mínimo):*** *Minimizar el costo presupuestario global de auditoría:*

$$\min_{Y \in \{0,1\}^n} \sum_{j=1}^{n} w_j y_j.$$

Definición 12 (Restricciones y Condición de Factibilidad) *El problema impone la cobertura integral de todos los riesgos del universo:*

$$\bigcup_{j:y_j=1} S_j = U \tag{3}$$

$$y_j \in \{0,1\}, \quad \forall j \in \{1, \ldots, n\}. \tag{4}$$

Para que exista al menos una solución factible, la instancia debe satisfacer la condición de cobertura global:

$$\bigcup_{j=1}^{n} S_j = U.$$

Si dicha unión es un subconjunto propio de U ($\bigcup_{j=1}^{n} S_j \subsetneq U$), *el problema es infactible y existen riesgos huérfanos imposibles de mitigar con el catálogo disponible.*

3.2. Demostración Formal de NP-Dureza por Reducción desde Vertex Cover

El problema de Cobertura de Auditoría es isomórfico al problema fundamental de **Cobertura de Conjuntos (Set Cover)**. Para fundamentar su NP-dureza, presentamos una reducción formal en tiempo polinomial desde el problema de **Cobertura de Vértices (Vertex Cover)**, uno de los 21 problemas NP-completos originales de Karp (1972).

Definición 13 (Problema de Decisión de Cobertura de Vértices (Vertex Cover)) *Dado un grafo no dirigido* $G = (V, E)$, *con vértices* $V = \{v_1, \ldots, v_p\}$ *y aristas* $E = \{e_1, \ldots, e_q\}$, *y un entero positivo* $k \leq |V|$: *¿existe un subconjunto de vértices* $V' \subseteq V$ *con* $|V'| \leq k$ *tal que para toda arista* $\{u, v\} \in E$ *se cumpla que* $\{u, v\} \cap V' \neq \emptyset$*?*

Definición 14 (Problema de Decisión de Cobertura de Auditoría (Audit Cover - D)) *Dado un universo de riesgos* U, *una familia de interacciones* $\mathcal{S} = \{S_1, \ldots, S_n\}$ *y un entero* k: *¿existe una subcolección* $\mathcal{C} \subseteq \mathcal{S}$ *con* $|\mathcal{C}| \leq k$ *tal que* $\bigcup_{S \in \mathcal{C}} S = U$*?*

Teorema 3 *El problema de Cobertura de Auditoría (Audit Cover - D) es NP-completo, y su versión de optimización es NP-difícil.*

Demostración:

1. Pertenencia a NP: Dado un certificado $\mathcal{C} \subseteq \mathcal{S}$:

- Verificar que $|\mathcal{C}| \leq k$ toma tiempo $\mathcal{O}(|\mathcal{C}|) \leq \mathcal{O}(n)$.
- Realizar la unión de los subconjuntos $\bigcup_{S_j \in \mathcal{C}} S_j$ e inspeccionar si contiene cada elemento de U toma tiempo $\mathcal{O}\left(\sum_{S_j \in \mathcal{C}} |S_j|\right) \leq \mathcal{O}(n \cdot r)$.

Como la verificación se ejecuta en tiempo estrictamente polinomial $\mathcal{O}(n \cdot r)$, Audit Cover - D $\in$ NP.

2. Reducción Polinomial (Vertex Cover $\leq_P$ Audit Cover - D):
Dada una instancia arbitraria de Vertex Cover $(G = (V, E), k)$, definimos la instancia correspondiente de Audit Cover:

1. **Universo de riesgos:** Identificamos cada arista del grafo como un riesgo normativo único. Así, el universo es $U = E$, con cardinalidad $r = |E| = q$.
2. **Interacciones candidatas:** Para cada vértice $v \in V$, definimos una interacción candidata S_v compuesta por todas las aristas incidentes en v:

   $$S_v = \{e \in E \mid v \in e\}.$$

   Por consiguiente, el número de interacciones es $n = |V| = p$, y la familia es $\mathcal{S} = \{S_v \mid v \in V\}$.
3. **Cota de cardinalidad:** El entero k permanece idéntico.

3. Complejidad Computacional de la Reducción:
Para cada vértice $v \in V$, identificar sus aristas incidentes requiere recorrer la lista de adyacencia del grafo. El tiempo requerido para generar todos los subconjuntos S_v es $\mathcal{O}(|V| + |E|)$. Esta transformación es lineal en el tamaño del grafo y, por ende, puramente polinomial.

4. Demostración de Equivalencia (Doble Implicación):

($\Rightarrow$) Supongamos que G tiene un Vertex Cover $V' \subseteq V$ con $|V'| \leq k$.
Consideremos la subcolección de interacciones $\mathcal{C} = \{S_v \mid v \in V'\}$. Claramente, $|\mathcal{C}| = |V'| \leq k$.
Sea $e = \{u, v\} \in E$ una arista arbitraria del universo U. Como V' es un Vertex Cover de G, al menos uno de los extremos de la arista pertenece a V'; supongamos sin pérdida de generalidad que $u \in V'$.
Por construcción, la interacción S_u incluye a e ($e \in S_u$). Como $S_u \in \mathcal{C}$, se deduce que:

$$e \in \bigcup_{S \in \mathcal{C}} S.$$

Como esto es válido para toda arista $e \in E = U$, concluimos que $\bigcup_{S \in \mathcal{C}} S = U$. Por consiguiente, $\mathcal{C}$ es una cobertura de auditoría válida de tamaño $\leq k$.

($\Leftarrow$) Supongamos que existe una cobertura de auditoría $\mathcal{C} \subseteq \mathcal{S}$ con $|\mathcal{C}| \leq k$ tal que $\bigcup_{S \in \mathcal{C}} S = U$.
Definimos el conjunto de vértices:

$$V' = \{v \in V \mid S_v \in \mathcal{C}\}.$$

Dado que cada interacción en $\mathcal{C}$ proviene unívocamente de un vértice de V, $|V'| = |\mathcal{C}| \leq k$.
Sea $e = \{u, v\} \in E = U$ una arista cualquiera. Puesto que $\mathcal{C}$ cubre U, existe al menos una interacción $S_w \in \mathcal{C}$ tal que $e \in S_w$.
Por definición de S_w, el vértice w debe ser uno de los extremos incidentes de la arista e, lo que implica necesariamente que $w = u$ o $w = v$.
Dado que $S_w \in \mathcal{C}$, tenemos que $w \in V'$. Así, al menos uno de los extremos de e pertenece a V', demostrando que la arista e está cubierta.
Siendo esto cierto para toda arista en E, V' es un Vertex Cover para el grafo G con $|V'| \leq k$.

Habiendo demostrado validez, bicondicionalidad y tiempo polinomial, queda demostrado que la Cobertura de Auditoría es NP-completa y su versión de optimización es NP-difícil. ■

3.3. Diseño del Algoritmo Voraz y Variante Ponderada

Al ser un problema NP-difícil, encontrar la solución exacta óptima requeriría tiempo exponencial. Se adopta la heurística voraz (*Greedy Set Cover*), formalizada por Johnson (1974), Lovász (1975) y Chvátal (1979).

3.3.1. Criterios de Selección Voraz

Sea $U_{restante}$ el conjunto de riesgos normativos que aún no han sido cubiertos en el transcurso del algoritmo. Inicialmente, $U_{restante} = U$.

- **Caso Estándar (No Ponderado):** En cada iteración k, seleccionar la interacción S^* que maximice la ganancia marginal de cobertura:

$$S^* = \arg \max_{S_j \in \mathcal{S}_{disp}} |S_j \cap U_{restante}|.$$

- **Variante 1 (Ponderada por Costo - Chvátal 1979):** En cada iteración k, seleccionar la interacción S^* que minimice el ratio costo-efectividad (costo por cada riesgo marginal cubierto):

$$S^* = \arg \min_{\substack{S_j \in \mathcal{S}_{disp} \\ |S_j \cap U_{restante}| > 0}} \frac{w_j}{|S_j \cap U_{restante}|}.$$

3.4. Deducción Paso a Paso de la Función de Eficiencia Asintótica

Teorema 4 *El Algoritmo Voraz de Cobertura de Auditoría tiene una cota superior de complejidad temporal en el peor caso de:*

$$T(n, r) = \mathcal{O}(n \cdot r^2)$$

donde $n = |\mathcal{S}|$ es el número de interacciones y $r = |U|$ es el número de categorías de riesgo.

Demostración:

Analizamos el cómputo paso a paso contando las operaciones elementales:

1. **Número máximo de iteraciones del bucle principal:**
   En cada iteración en la que se selecciona un subconjunto factible, se cubre al menos un nuevo

elemento de $U_{restante}$ ($|S^* \cap U_{restante}| \geq 1$). Como la cardinalidad inicial es $|U| = r$, el número de iteraciones está acotado estrictamente por:

$$K \leq r.$$

El peor caso ocurre cuando cada interacción seleccionada aporta exactamente un único elemento nuevo al conjunto de riesgos cubiertos, ejecutándose exactamente r iteraciones.

2. **Costo de evaluación por iteración:**
   En cada iteración del bucle principal:
   - Se evalúan a lo sumo las n interacciones candidatas disponibles.
   - Para cada interacción S_j, se calcula la intersección $S_j \cap U_{restante}$. Utilizando tablas hash o conjuntos estándar con operaciones de pertenencia $\mathcal{O}(1)$, el costo de calcular la intersección es $\mathcal{O}(|S_j|) \leq \mathcal{O}(r)$.
   - La división $w_j/|nuevos|$ y la comparación de mínimos toma tiempo constante $\mathcal{O}(1)$.
   - Por consiguiente, evaluar todos los candidatos en una iteración toma:

$$T_{eval} = \sum_{j=1}^{n} \mathcal{O}(|S_j|) \leq n \cdot \mathcal{O}(r) = \mathcal{O}(n \cdot r).$$

3. **Costo de actualización del estado:**
   Una vez determinado el mejor candidato, remover los elementos cubiertos $U_{restante} \leftarrow U_{restante} \setminus mejores_nuevos$ requiere $\mathcal{O}(|mejores_nuevos|) \leq \mathcal{O}(r)$ operaciones. La inserción en la traza y en la lista de solución toma tiempo $\mathcal{O}(1)$.

4. **Complejidad total acumulada:**
   Sumando sobre las a lo sumo r iteraciones:

$$T(n, r) = \sum_{k=1}^{r} \Big(\mathcal{O}(n \cdot r) + \mathcal{O}(r) \Big) = \sum_{k=1}^{r} \mathcal{O}(n \cdot r) = \mathcal{O}(n \cdot r^2).$$

■

3.5. Demostración y Análisis de la Cota Armónica de Aproximación

A pesar de ser una heurística voraz para un problema NP-difícil, el algoritmo posee una garantía formal de aproximación reconocida como una de las cotas teóricas más célebres de la optimización combinatoria.

Definición 15 (Número Armónico) *El r-ésimo número armónico, denotado por H_r, se define como:*

$$H_r = \sum_{k=1}^{r} \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{r}.$$

Por propiedades analíticas del cálculo integral (comparación con $\int_1^r \frac{1}{x} dx$):

$$\ln(r) < H_r \leq \ln(r) + 1.$$

Teorema 5 (Garantía de Aproximación de Johnson (1974), Lovász (1975) y Chvátal (1979)) *Sean OPT el costo de la solución óptima del problema de Cobertura de Auditoría (sea cardinal o ponderado) y SOL_{voraz} el costo obtenido por el Algoritmo Voraz. Entonces:*

$$SOL_{voraz} \leq H_r \cdot OPT \leq (\ln(r) + 1) \cdot OPT$$

donde $r = |U|$ (o más precisamente, $r = \max_j |S_j|$).

Demostración:

Distribuimos el costo de seleccionar cada interacción entre los nuevos elementos que dicha interacción cubre. Cuando el algoritmo voraz selecciona el subconjunto S_j de costo w_j, y este cubre un conjunto de nuevos elementos $C = S_j \cap U_{restante}$, asignamos a cada elemento recién cubierto $e \in C$ un costo marginal individual:

$$\text{costo}(e) = \frac{w_j}{|S_j \cap U_{restante}|}.$$

Por construcción, la suma de los costos individuales asignados a todos los elementos del universo es exactamente igual al costo total de la solución voraz:

$$SOL_{voraz} = \sum_{e \in U} \text{costo}(e).$$

Consideremos ahora la solución óptima OPT, que consiste en una familia de conjuntos $\mathcal{C}^* \subseteq \mathcal{S}$ tal que $\bigcup_{S^* \in \mathcal{C}^*} S^* = U$ y $\sum_{S^* \in \mathcal{C}^*} w(S^*) = OPT$.
Reescribiendo la suma:

$$SOL_{voraz} \leq \sum_{S^* \in \mathcal{C}^*} \sum_{e \in S^*} \text{costo}(e).$$

Fijemos cualquier conjunto óptimo $S^* \in \mathcal{C}^*$ de tamaño $k = |S^*|$. Sean $e_1, e_2, \ldots, e_k$ los elementos de S^* indexados en el orden exacto en el que fueron cubiertos por el algoritmo voraz.
En el instante inmediatamente previo a que el elemento e_m ($1 \leq m \leq k$) fuera cubierto, al menos los elementos $\{e_m, e_{m+1}, \ldots, e_k\}$ (en total $k - m + 1$ elementos de S^*) permanecían descubiertos. En ese momento, el subconjunto S^* estaba disponible como candidato y podía cubrir al menos $k - m + 1$ elementos con costo $w(S^*)$.
Dado que el algoritmo voraz eligió la interacción con el ratio mínimo:

$$\text{costo}(e_m) \leq \frac{w(S^*)}{k - m + 1}.$$

Sumando esta cota sobre todos los elementos $e_m \in S^*$:

$$\sum_{e \in S^*} \text{costo}(e) = \sum_{m=1}^{k} \text{costo}(e_m) \leq \sum_{m=1}^{k} \frac{w(S^*)}{k - m + 1} = w(S^*) \sum_{j=1}^{k} \frac{1}{j} = w(S^*) \cdot H_{|S^*|}.$$

Dado que $|S^*| \leq r$, tenemos $H_{|S^*|} \leq H_r$. Sumando sobre todos los subconjuntos $S^* \in \mathcal{C}^*$:

$$SOL_{voraz} \leq \sum_{S^* \in \mathcal{C}^*} w(S^*) \cdot H_r = H_r \sum_{S^* \in \mathcal{C}^*} w(S^*) = H_r \cdot OPT.$$

■

Corolario 2 (Optimalidad Asintótica de la Cota) *Feige (1998) demostró que, a menos que $P = NP$, ningún algoritmo en tiempo polinomial puede aproximar Set Cover dentro de un factor de $(1 - \epsilon)\ln(r)$ para cualquier $\epsilon > 0$. Por lo tanto, la cota de aproximación del algoritmo voraz implementado en este proyecto es teóricamente inmejorable en el sentido asintótico.*

4. Resultados sobre el Caso de Prueba Oficial

4.1. Descripción del Caso de Prueba

El caso oficial proporcionado por la profesora corresponde al archivo `entrada_G3_mochila_20_nodos.json`, con los siguientes parámetros de instancia:

- $n = 20$ etapas en serie (E01 – E20), con flujo lineal $E01 \to E02 \to \cdots \to E20$.
- Presupuesto de aceleración: $B = 72$ unidades.
- Universo de riesgos de auditoría: $r = 12$ categorías (sesgo, privacidad, formato, seguridad, trazabilidad, equidad, coherencia, robustez, disponibilidad, cumplimiento, integridad, transparencia).
- Interacciones candidatas de auditoría: $n_{audit} = 35$ variaciones (una parcial y una completa por cada etapa, con excepción de E20 que cuenta con variaciones adicionales).
- Presupuesto conjunto (aceleración + auditoría): $B_{conjunto} = 100$ unidades.
- Duración nominal total de la cadena: $T_{nominal} = \sum_{i=1}^{20} t_i = 501$ unidades.

4.2. Resultados: Aceleración Óptima (DP Mochila 0/1)

La solución óptima obtenida por programación dinámica sobre la instancia con $n = 20$ y $B = 72$ es la siguiente:

Etapa	**Costo (c_i)**	**Reducción (d_i)**	**Decisión**
E02	12	9	ACELERADA
E03	5	4	ACELERADA
E04	16	12	ACELERADA
E06	13	10	ACELERADA
E08	17	13	ACELERADA
E12	9	7	ACELERADA
Total	**72**	**55**	

Métrica	**Valor**
Duración nominal total	501 unidades
Ahorro óptimo logrado	55 unidades
Duración final reducida	446 unidades
Presupuesto utilizado	72 / 72 (100 %)
Tiempo de cómputo DP	≈ 0.2242 ms

4.3. Resultados: Cobertura Mínima de Auditoría

4.3.1. Voraz Estándar (Cardinalidad Mínima)

Paso	Interacción	Costo	Riesgos cubiertos	Quedan
1	E01_completa	6.00	R01, R02, R03, R10	8
2	E16_completa	7.00	R04, R05, R09, R12	4
3	E06_completa	8.00	R07, R08, R11	1
4	E03_completa	8.00	R06	0
Total		**29.00**	**12 / 12**	

4.3.2. Voraz Ponderado por Costo (Variante 1 – Chvátal 1979)

Paso	Interacción	Costo	Ratio	Riesgos cubiertos	Quedan
1	E01_completa	6.00	1.5000	R01, R02, R03, R10	8
2	E16_completa	7.00	1.7500	R04, R05, R09, R12	4
3	E06_completa	8.00	2.6667	R07, R08, R11	1
4	E03_parcial	5.00	5.0000	R06	0
Total		**26.00**		**12 / 12**	

La variante ponderada selecciona `E03_parcial` en lugar de `E03_completa` en el paso 4 (costo 5 vs. 8), reduciendo el costo total de auditoría en 3 unidades (10.3 % de ahorro).

4.4. Verificación del Presupuesto Conjunto

Combinación	C. Aceleración	C. Auditoría	Total	Estado ($B_{conj} = 100$)
Estándar	72	29.00	101.00	EXCEDE
Ponderada	72	26.00	98.00	CUMPLE (margen: 2)

La variante ponderada es la única combinación que satisface el presupuesto conjunto de 100 unidades, con un margen de 2 unidades.

4.5. Resumen de Tiempos de Ejecución

Algoritmo	Tiempo (ms)
Programación Dinámica (Mochila 0/1)	≈ 0.2242 ms
Voraz Estándar (Set Cover Cardinal)	≈ 0.1741 ms
Voraz Ponderado (Variante 1)	≈ 0.2854 ms
Total	≈ 0.6837 ms

Todos los tiempos fueron medidos mediante `time.perf_counter()` de Python 3.13 sobre hardware de desarrollo estándar. La solución completa para $n = 20$ etapas, $B = 72$ y $r = 12$ riesgos se ejecuta en menos de 1 ms, confirmando la viabilidad práctica del enfoque pseudo-polinomial para instancias de tamaño razonable.

5. Arquitectura del Repositorio y Trazabilidad de Resultados

5.1. Separación Estricta de Responsabilidades (SoC)

El código fuente en el repositorio del equipo (`Proyecto-ADA`) ha sido estructurado respetando estrictamente los principios de ingeniería de software moderna y Clean Code:

- **Capa de Dominio y Modelos (`src/models/`):** Dataclasses fuertemente tipadas (`Etapa`, `Interaccion`, `ResultadoAceleracion`, `ResultadoAuditoria`) con validaciones de invariantes en `__post_init__`.
- **Capa Algorítmica (`src/algorithms/`):** Implementación pura de lógica computacional desacoplada de la entrada/salida:
  - `aceleracion_dp.py`: Programación dinámica para Mochila 0/1 y reconstrucción con justificación paso a paso.
  - `auditoria_voraz.py`: Heurística voraz cardinal estándar con desempate determinista y detección de infactibilidad.
  - `auditoria_ponderada.py`: Heurística voraz ponderada de Chvátal con cálculo de ratios marginales.
- **Capa de Infraestructura y Utilidades (`src/utils/`):**
  - `io_handler.py`: Deserialización robusta y serialización JSON con encoder para conjuntos y dataclasses.
  - `timer.py`: Medición con `time.perf_counter()` de resolución en microsegundos.
- **Capa de Orquestación (`main.py`):** CLI con interfaz `argparse` para procesar archivos individuales o lotes de carpetas.

5.2. Estructura de Trazas Generadas

Cada ejecución produce un archivo en `data/resultados/[caso]_resultado.json` con cuatro bloques esenciales:

1. `tiempos_ejecucion_segundos`: Tiempos independientes para DP, Voraz estándar, Voraz ponderado y total.
2. `aceleracion_dp`: Ahorro óptimo, duración original, duración final reducida, presupuesto utilizado y el desglose de decisiones cronológicas justificadas por etapa.
3. `auditoria_voraz`: Cardinalidad lograda, lista ordenada de interacciones elegidas y traza iterativa con candidatos evaluados en cada paso.
4. `auditoria_ponderada`: Costo total minimizado, interacciones elegidas y traza con ratios calculados en cada iteración.

6. Conclusiones

El marco matemático presentado resuelve integralmente los requerimientos teóricos y prácticos de la Primera Entrega del Proyecto Semestral ADAG:

1. Se demostró que la aceleración de una cadena en serie bajo presupuesto es formalmente equivalente al problema de la Mochila 0/1, siendo NP-difícil y resoluble exactamente en tiempo pseudo-polinomial $\Theta(n \cdot B)$.
2. Se demostró que la cobertura mínima de auditoría es equivalente a Vertex Cover / Set Cover, siendo NP-difícil, y se fundamentó el algoritmo voraz en tiempo $\mathcal{O}(n \cdot r^2)$ con garantía armónica $H_r \leq \ln(r) + 1$, la cual es inmejorable bajo la hipótesis $P \neq NP$.
3. Se implementó una solución de software modular, reproducible, libre de valores fijos en código, con medición precisa y trazas estructuradas en formato JSON.
4. Sobre el caso oficial de la profesora ($n = 20$, $B = 72$, $r = 12$), el algoritmo DP halló el óptimo exacto (ahorro de 55 unidades, $T_{final} = 446$) en menos de 0.23 ms; el voraz ponderado logró cobertura completa con costo 26 (3 unidades menos que el estándar) en 0.29 ms, siendo la única combinación que cumple el presupuesto conjunto de 100 unidades.

Referencias Bibliográficas

- Chvátal, V. (1979). "A greedy heuristic for the set-covering problem". *Mathematics of Operations Research*, 4(3), 233–235.
- Feige, U. (1998). "A threshold of $\ln n$ for approximating set cover". *Journal of the ACM*, 45(4), 634–652.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co.
- Johnson, D. S. (1974). "Approximation algorithms for combinatorial problems". *Journal of Computer and System Sciences*, 9(3), 256–278.
- Karp, R. M. (1972). "Reducibility among combinatorial problems". En *Complexity of Computer Computations* (pp. 85–103). Springer US.
- Lovász, L. (1975). "On the ratio of optimal integral and fractional covers". *Discrete Mathematics*, 13(4), 383–390.